Série BD : GROUP BY – HAVING – Jointure – Sous-requêtes – Date

Le directeur du restaurant « Dar Zitouna »

4ème SI – Lycée Rafèha – Enseignant : Aymen Essouyah

1. La situation

Le directeur du restaurant Dar Zitouna veut comprendre sa journée de travail. Chaque ticket de caisse est une ligne de la table COMMANDE ; chaque serveur est une ligne de la table SERVEUR.

Schéma : `SERVEUR (num, nom, salle) – COMMANDE (num, plat, categorie, prix, qte, numserveur)`

prix est le prix unitaire en DT. numserveur est une clé étrangère : elle référence SERVEUR(num).

Table SERVEUR

num	nom	salle
1	Sami	Terrasse
2	Rim	Salon
3	Karim	Terrasse
4	Nour	Salon

Table COMMANDE

num	plat	categorie	prix	qte	numserveur
1	Salade tunisienne	Entrée	6	2	1
2	Brik à l'oeuf	Entrée	4	5	2
3	Couscous à l'agneau	Plat	22	3	1
4	Poisson grillé	Plat	28	2	3
5	Ojja merguez	Plat	18	4	2
6	Makroudh	Dessert	5	6	3
7	Assidat zgougou	Dessert	7	3	1
8	Thé à la menthe	Boisson	3	8	2
9	Jus d'orange	Boisson	4	5	3
10	Brik au thon	Entrée	4	4	1
11	Lablabi	Plat	9	2	2
12	Eau minérale	Boisson	2	10	3

2. Le chemin d'une requête

Imagine le directeur devant sa pile de tickets. Il traite toujours les clauses dans le même ordre, quel que soit l'ordre dans lequel tu les écris :

FROM tous les tickets → **WHERE** tickets gardés → **GROUP BY** piles → **HAVING** piles gardées → **SELECT** 1 ligne par pile

Les deux cases grisées sont les deux filtres : WHERE trie les tickets, HAVING trie les piles.

N°	Clause	Le directeur se demande…	Ce qu'il fait
1	`FROM`	*Quels tickets ai-je sur mon bureau ?*	Il sort la table (ou les tables) concernée(s).
2	`WHERE`	*Ce ticket m'intéresse-t-il ?*	Il jette, un par un, les tickets qui ne conviennent pas. La jointure table1.x = table2.x s'écrit ici. Jamais de COUNT, SUM, AVG…
3	`GROUP BY`	*Comment je range ce qui reste ?*	Il fait une pile par valeur (une pile par catégorie, par serveur, par salle…).
4	`HAVING`	*Cette pile m'intéresse-t-elle ?*	Il garde ou jette des piles entières, d'après un résultat : COUNT, SUM, AVG, MAX, MIN.
5	`SELECT`	*Que j'écris sur mon résumé ?*	Une ligne par pile (ou par ticket s'il n'y a pas de GROUP BY).
6	`ORDER BY`	*Dans quel ordre je présente ?*	Il trie le résumé.

Règles à retenir

- **WHERE** parle d'un ticket (un champ, une comparaison). **HAVING** parle d'une pile (une fonction d'agrégat).
- Tout champ du SELECT qui n'est pas dans une fonction (COUNT, SUM…) doit aussi être dans le GROUP BY.
- Une jointure s'écrit dans le WHERE : table1.x = table2.x.
- On filtre d'abord les tickets (WHERE), puis on fait les piles (GROUP BY), puis on filtre les piles (HAVING).

Méthode en 4 questions

1. Que veut-on afficher, et une ligne pour quoi ? (la réponse après « pour chaque » donne le GROUP BY)
2. Quel calcul fait-on sur chaque pile ? (COUNT, SUM, AVG, MAX, MIN)
3. Y a-t-il une condition sur les tickets, avant de faire les piles ? (WHERE)
4. Y a-t-il une condition sur le résultat d'une pile ? (HAVING) – et faut-il une 2e table (jointure) ou une sous-requête ?

3. Exercices

Écris chaque requête dans le cadre prévu. Fais d'abord les piles « dans ta tête » avec les tableaux ci-dessus.

Partie A – Une seule table : COMMANDE

A1. Afficher, pour chaque catégorie de plat, le nombre de commandes.

Réponse :

A2. Afficher, pour chaque catégorie, la quantité totale commandée.

Réponse :

A3. Le chiffre d'affaires d'une commande est prix × qte. Afficher le chiffre d'affaires de chaque catégorie.

Réponse :

A4. Afficher les catégories dont le chiffre d'affaires dépasse 100 DT.

Réponse :

A5. Parmi les commandes dont le prix unitaire est supérieur ou égal à 5 DT, afficher le nombre de commandes de chaque catégorie.

Réponse :

A6. Parmi les commandes dont le prix unitaire est supérieur ou égal à 4 DT, afficher les catégories qui ont au moins 3 commandes, avec ce nombre.

Réponse :

A7. MariaDB refuse la requête suivante. Expliquer pourquoi, puis la corriger pour afficher les catégories ayant plus de 2 commandes.

```
SELECT categorie, COUNT(*)
FROM COMMANDE
WHERE COUNT(*) > 2
GROUP BY categorie;
```

Réponse :

Partie B – Deux tables : COMMANDE et SERVEUR (jointure)

B1. Afficher le nom de chaque serveur et le chiffre d'affaires des commandes qu'il a prises.

Réponse :

B2. Afficher les serveurs dont le chiffre d'affaires dépasse 120 DT.

Réponse :

B3. Afficher, pour chaque salle (Terrasse, Salon), le nombre de commandes prises par ses serveurs.

Réponse :

B4. Parmi les commandes de la salle Terrasse qui ne sont pas des boissons, afficher les serveurs qui ont pris au moins 3 commandes, avec ce nombre.

Réponse :

B5. Pour la requête de la question B4, complète le tableau : combien de lignes (ou de piles) reste-t-il à chaque étape ?

Étape	Nombre
Tickets de COMMANDE associés à leur serveur (jointure)	
Tickets restants après : salle = 'Terrasse' et catégorie <> 'Boisson'	
Piles formées par GROUP BY	
Piles gardées par HAVING COUNT(*) >= 3	

Partie C – Sous-requêtes

C1. Afficher le serveur dont le chiffre d'affaires est le plus élevé.

Réponse :

C2. Afficher le nom des serveurs qui n'ont pris aucune commande.

Réponse :

C3. Afficher les plats (nom et prix) dont le prix unitaire est supérieur à la moyenne des prix de toutes les commandes.

Réponse :

C4. Afficher les catégories dont le chiffre d'affaires est supérieur à la moyenne des chiffres d'affaires des catégories.

Réponse :

Partie D – Fonctions sur les dates

Rappel des fonctions :

• DATE(chaine) : extrait une date d'une chaîne au format DATE/DATETIME.	• DAY(date) : retourne le jour.	• MONTH(date) : retourne le mois.
• YEAR(date) : retourne l'année.	• NOW() : retourne la date et l'heure courantes.	• ADDDATE(date, n) : ajoute n jours (ou un intervalle) à une date.
• DATEDIFF(date1, date2) : différence en jours entre deux dates.		

Questions

D1. Afficher les commandes passées le 15 mars 2026.

D2. Afficher, pour chaque commande, le numéro, le plat et le jour du mois de la commande.

D3. Afficher les commandes passées au mois de mars 2026

D4. Afficher, pour chaque année, le nombre de commandes.

D5. Afficher les commandes passées dans les 7 derniers jours par rapport à la date du jour (on suppose que la date du jour est le 15 avril 2027).

D6. Afficher, pour chaque commande, le nombre de jours écoulés depuis la commande jusqu'au 31 décembre 2027.

D7. Afficher les serveurs embauchés depuis plus de 5 ans par rapport au 1er janvier 2027.

D8. Afficher, pour chaque serveur, son nom et le nombre de commandes qu'il a prises au mois de mars 2026.

D9. Afficher les catégories dont le chiffre d'affaires réalisé entre le 1er janvier 2027 et le 30 avril 2027 dépasse 50 DT.

D10. Expliquer pourquoi la requête suivante est refusée, puis la corriger.

```
SELECT num, plat, DAY(date_cmd)
FROM COMMANDE
WHERE DAY(date_cmd) = 15
GROUP BY YEAR(date_cmd);
```

D11. Afficher les commandes dont la date est comprise entre le 1er janvier 2026 et le 31 décembre 2026, triées par date croissante.

D12. Afficher, pour chaque serveur, la date de sa première commande et le nombre de jours entre cette première commande et le 31 décembre 2027.

D13. Afficher les commandes passées au cours du premier trimestre 2027 (janvier, février, mars).

D14. Afficher le chiffre d'affaires par mois pour l'année 2026.

D15. Afficher les serveurs dont la date d'embauche est postérieure au 1er janvier 2023, avec le nombre d'années écoulées jusqu'au 1er janvier 2027.

Tableau récapitulatif des fonctions de date

Fonction	Rôle	Exemple
DATE()	Extrait une date d'une chaîne	DATE('2026-03-15 10:00:00') →
DAY()	Jour du mois	DAY('2026-03-15') →
MONTH()	Mois	MONTH('2026-03-15') →
YEAR()	Année	YEAR('2026-03-15') →
NOW()	Date et heure courantes	NOW()
ADDDATE()	Ajoute des jours	ADDDATE('2026-03-15', 7) →
DATEDIFF()	Différence en jours	DATEDIFF('2026-03-22', '2026-03-15') →

– Corrigé –

Série BD : GROUP BY – HAVING – Jointure – Sous-requêtes – Date (Dar Zitouna). Tous les résultats ont été obtenus en exécutant les requêtes sur MariaDB 10.11.

Script de création et d'insertion

```
CREATE TABLE SERVEUR (
  num INT PRIMARY KEY,
  nom VARCHAR(20),
  salle VARCHAR(20)
);
CREATE TABLE COMMANDE (
  num INT PRIMARY KEY,
  plat VARCHAR(30),
  categorie VARCHAR(15),
  prix DECIMAL(5,2),
  qte INT,
  numserveur INT,
  FOREIGN KEY (numserveur) REFERENCES SERVEUR(num)
);
INSERT INTO SERVEUR VALUES
  (1,'Sami','Terrasse'), (2,'Rim','Salon'),
  (3,'Karim','Terrasse'), (4,'Nour','Salon');
INSERT INTO COMMANDE VALUES
  (1,'Salade tunisienne','Entrée',6,2,1),
  (2,'Brik à l''oeuf','Entrée',4,5,2),
  (3,'Couscous à l''agneau','Plat',22,3,1),
  (4,'Poisson grillé','Plat',28,2,3),
  (5,'Ojja merguez','Plat',18,4,2),
  (6,'Makroudh','Dessert',5,6,3),
  (7,'Assidat zgougou','Dessert',7,3,1),
  (8,'Thé à la menthe','Boisson',3,8,2),
  (9,'Jus d''orange','Boisson',4,5,3),
  (10,'Brik au thon','Entrée',4,4,1),
  (11,'Lablabi','Plat',9,2,2),
  (12,'Eau minérale','Boisson',2,10,3);
```

Correction détaillée

Partie A – Une seule table : COMMANDE

A1 – Compter par catégorie

Afficher, pour chaque catégorie de plat, le nombre de commandes.

Requête :

```
SELECT categorie, COUNT(*) AS nb
FROM COMMANDE
GROUP BY categorie;
```

Résultat :

categorie	nb
Boisson	3
Dessert	2
Entrée	3
Plat	4

Autre façon : COUNT(num) donne le même résultat ici

```
SELECT categorie, COUNT(num) AS nb
FROM COMMANDE
GROUP BY categorie;
```

Même résultat.

COUNT(champ) ignore les valeurs NULL ; num est la clé primaire, donc jamais NULL.

À expliquer : Une ligne par pile (catégorie). Le champ du SELECT qui n'est pas une fonction est dans le GROUP BY.

A2 – Additionner par catégorie

Afficher, pour chaque catégorie, la quantité totale commandée.

Requête :

```
SELECT categorie, SUM(qte) AS total_qte
FROM COMMANDE
GROUP BY categorie;
```

Résultat :

categorie	total_qte
Boisson	23
Dessert	9
Entrée	11
Plat	11

À expliquer : Erreur fréquente : écrire COUNT(qte) au lieu de SUM(qte). COUNT compte les tickets, SUM additionne les quantités.

A3 – Calculer avant d'additionner

Le chiffre d'affaires d'une commande est prix × qte. Afficher le chiffre d'affaires de chaque catégorie.

Requête :

```
SELECT categorie, SUM(prix*qte) AS ca
FROM COMMANDE
GROUP BY categorie;
```

Résultat :

categorie	ca
Boisson	64.00
Dessert	51.00
Entrée	48.00
Plat	212.00

À expliquer : Le calcul prix*qte se fait ticket par ticket, puis SUM additionne les résultats de la pile.

A4 – HAVING : filtrer les piles

Afficher les catégories dont le chiffre d'affaires dépasse 100 DT.

Requête :

```
SELECT categorie, SUM(prix*qte) AS ca
FROM COMMANDE
GROUP BY categorie
HAVING SUM(prix*qte) > 100;
```

Résultat :

categorie	ca
Plat	212.00

Autre façon : Avec l'alias dans HAVING

```
SELECT categorie, SUM(prix*qte) AS ca
FROM COMMANDE
GROUP BY categorie
HAVING ca > 100;
```

Même résultat.

Accepté par MariaDB/MySQL, mais pas par tous les SGBD : on recommande de répéter la fonction.

À expliquer : La condition porte sur le résultat d'une fonction de groupe : elle va dans HAVING, jamais dans WHERE.

A5 – WHERE : filtrer les tickets

Parmi les commandes dont le prix unitaire est supérieur ou égal à 5 DT, afficher le nombre de commandes de chaque catégorie.

Requête :

```
SELECT categorie, COUNT(*) AS nb
FROM COMMANDE
WHERE prix >= 5
GROUP BY categorie;
```

Résultat :

categorie	nb
Dessert	2
Entrée	1
Plat	4

À expliquer : WHERE s'exécute avant GROUP BY : les tickets à moins de 5 DT sont jetés avant de faire les piles. La catégorie Boisson disparaît car aucun de ses tickets ne reste.

A6 – WHERE + GROUP BY + HAVING

Parmi les commandes dont le prix unitaire est supérieur ou égal à 4 DT, afficher les catégories qui ont au moins 3 commandes, avec ce nombre.

Requête :

```
SELECT categorie, COUNT(*) AS nb
FROM COMMANDE
WHERE prix >= 4
GROUP BY categorie
HAVING COUNT(*) >= 3;
```

Résultat :

categorie	nb
Entrée	3
Plat	4

À expliquer : Deux conditions, deux endroits : prix >= 4 parle d'un ticket (WHERE), COUNT(*) >= 3 parle d'une pile (HAVING). Écrire prix >= 4 dans HAVING est une erreur : prix n'existe plus une fois les piles formées.

A7 – Prévoir l'erreur

MariaDB refuse la requête suivante. Expliquer pourquoi, puis la corriger pour afficher les catégories ayant plus de 2 commandes.

```
SELECT categorie, COUNT(*)
FROM COMMANDE
WHERE COUNT(*) > 2
GROUP BY categorie;
```

Résultat : `ERROR 1111 (HY000) : Invalid use of group function`

Requête corrigée :

```
SELECT categorie, COUNT(*)
FROM COMMANDE
GROUP BY categorie
HAVING COUNT(*) > 2;
```

Résultat :

categorie	COUNT(*)
Boisson	3
Entrée	3
Plat	4

À expliquer : Au moment du WHERE, les piles n'existent pas encore : COUNT(*) n'a pas de sens. Message obtenu : « Invalid use of group function ».

Partie B – Deux tables : COMMANDE et SERVEUR (jointure)

B1 – Jointure + GROUP BY

Afficher le nom de chaque serveur et le chiffre d'affaires des commandes qu'il a prises.

Requête :

```
SELECT SERVEUR.nom, SUM(COMMANDE.prix*COMMANDE.qte) AS ca
FROM COMMANDE, SERVEUR
WHERE COMMANDE.numserveur = SERVEUR.num
GROUP BY SERVEUR.num, SERVEUR.nom;
```

Résultat :

nom	ca
Sami	115.00
Rim	134.00
Karim	126.00

Autre façon : Avec une sous-requête corrélée dans le SELECT

```
SELECT nom, (SELECT SUM(prix*qte)
             FROM COMMANDE
             WHERE COMMANDE.numserveur = SERVEUR.num) AS ca
FROM SERVEUR;
```

nom	ca
Sami	115.00
Rim	134.00
Karim	126.00
Nour	NULL

Différence : Nour (aucune commande) apparaît avec NULL. La jointure l'élimine.

À expliquer : La jointure COMMANDE.numserveur = SERVEUR.num fait partie du WHERE (étape 1 : on associe chaque ticket à son serveur). Nour n'apparaît pas : aucun ticket ne lui correspond.

B2 – Jointure + HAVING

Afficher les serveurs dont le chiffre d'affaires dépasse 120 DT.

Requête :

```
SELECT SERVEUR.nom, SUM(COMMANDE.prix*COMMANDE.qte) AS ca
FROM COMMANDE, SERVEUR
WHERE COMMANDE.numserveur = SERVEUR.num
GROUP BY SERVEUR.num, SERVEUR.nom
HAVING SUM(COMMANDE.prix*COMMANDE.qte) > 120;
```

Résultat :

nom	ca
Rim	134.00
Karim	126.00

À expliquer : La jointure reste dans WHERE ; la condition sur le chiffre d'affaires va dans HAVING.

B3 – Grouper sur la 2e table

Afficher, pour chaque salle (Terrasse, Salon), le nombre de commandes prises par ses serveurs.

Requête :

```
SELECT SERVEUR.salle, COUNT(*) AS nb
FROM COMMANDE, SERVEUR
WHERE COMMANDE.numserveur = SERVEUR.num
GROUP BY SERVEUR.salle;
```

Résultat :

salle	nb
Salon	4
Terrasse	8

À expliquer : On peut grouper sur un champ de n'importe quelle table de la jointure.

B4 – Jointure + WHERE + GROUP BY + HAVING

Parmi les commandes de la salle Terrasse qui ne sont pas des boissons, afficher les serveurs qui ont pris au moins 3 commandes, avec ce nombre.

Requête :

```
SELECT SERVEUR.nom, COUNT(*) AS nb
FROM COMMANDE, SERVEUR
WHERE COMMANDE.numserveur = SERVEUR.num
  AND SERVEUR.salle = 'Terrasse'
  AND COMMANDE.categorie <> 'Boisson'
GROUP BY SERVEUR.num, SERVEUR.nom
HAVING COUNT(*) >= 3;
```

Résultat :

nom	nb
Sami	4

Autre façon : Avec IN à la place de la jointure (on obtient le numéro du serveur)

```
SELECT numserveur, COUNT(*) AS nb
FROM COMMANDE
WHERE numserveur IN (SELECT num FROM SERVEUR WHERE salle = 'Terrasse')
  AND categorie <> 'Boisson'
GROUP BY numserveur
HAVING COUNT(*) >= 3;
```

numserveur	nb
1	4

Pour afficher le nom il faut la jointure : IN ne permet pas d'afficher un champ de SERVEUR.

À expliquer : Toutes les conditions sur les tickets (jointure, salle, catégorie) sont dans WHERE ; seule la condition sur le nombre est dans HAVING.

B5 – Les étapes de B4

Étape	Nombre
Tickets de COMMANDE associés à leur serveur (jointure)	12
Tickets restants après : salle = 'Terrasse' et catégorie <> 'Boisson'	6
Piles formées par GROUP BY	2 (Sami, Karim)
Piles gardées par HAVING COUNT(*) >= 3	1 (Sami)

Sami : tickets 1, 3, 7, 10 (4 tickets). Karim : tickets 4 et 6 (2 tickets), car ses tickets 9 et 12 sont des boissons. Rim et Nour travaillent au Salon : leurs tickets sont éliminés par WHERE.

Partie C – Sous-requêtes

C1 – Sous-requête : le maximum

Afficher le serveur dont le chiffre d'affaires est le plus élevé.

Requête :

```
SELECT SERVEUR.nom, SUM(COMMANDE.prix*COMMANDE.qte) AS ca
FROM COMMANDE, SERVEUR
WHERE COMMANDE.numserveur = SERVEUR.num
GROUP BY SERVEUR.num, SERVEUR.nom
HAVING SUM(COMMANDE.prix*COMMANDE.qte) >= ALL
   (SELECT SUM(prix*qte)
    FROM COMMANDE
    GROUP BY numserveur);
```

Résultat :

nom	ca
Rim	134.00

Autre façon : Avec MAX sur une table dérivée

```
SELECT SERVEUR.nom, SUM(COMMANDE.prix*COMMANDE.qte) AS ca
FROM COMMANDE, SERVEUR
WHERE COMMANDE.numserveur = SERVEUR.num
GROUP BY SERVEUR.num, SERVEUR.nom
HAVING SUM(COMMANDE.prix*COMMANDE.qte) =
   (SELECT MAX(t.ca)
    FROM (SELECT SUM(prix*qte) AS ca
          FROM COMMANDE
          GROUP BY numserveur) AS t);
```

Même résultat.

La table dérivée du FROM doit avoir un alias (ici t).

Autre façon : Avec ORDER BY et LIMIT (sans sous-requête)

```
SELECT SERVEUR.nom, SUM(COMMANDE.prix*COMMANDE.qte) AS ca
FROM COMMANDE, SERVEUR
WHERE COMMANDE.numserveur = SERVEUR.num
GROUP BY SERVEUR.num, SERVEUR.nom
ORDER BY ca DESC
LIMIT 1;
```

Même résultat.

Attention : en cas d'égalité entre deux serveurs, LIMIT 1 n'en affiche qu'un seul.

À expliquer : La sous-requête calcule le chiffre d'affaires de chaque serveur ; la requête principale garde celui qui est supérieur ou égal à tous les autres.

C2 – Sous-requête : l'absent

Afficher le nom des serveurs qui n'ont pris aucune commande.

Requête :

```
SELECT nom
FROM SERVEUR
WHERE num NOT IN (SELECT numserveur
                  FROM COMMANDE);
```

Résultat :

nom
Nour

Autre façon : Avec NOT EXISTS

```
SELECT nom
FROM SERVEUR
WHERE NOT EXISTS (SELECT *
                  FROM COMMANDE
                  WHERE COMMANDE.numserveur = SERVEUR.num);
```

Même résultat.

NOT EXISTS reste correct même si numserveur contenait des NULL ; NOT IN ne renverrait alors aucune ligne.

À expliquer : Une jointure ne peut pas répondre : elle ne garde que les serveurs qui ont au moins un ticket.

C3 – Sous-requête : la moyenne

Afficher les plats (nom et prix) dont le prix unitaire est supérieur à la moyenne des prix de toutes les commandes.

Requête :

```
SELECT plat, prix
FROM COMMANDE
WHERE prix > (SELECT AVG(prix)
              FROM COMMANDE);
```

Résultat :

plat	prix
Couscous à l'agneau	22.00
Poisson grillé	28.00
Ojja merguez	18.00

À expliquer : La moyenne vaut 9,33 DT (valeur de AVG(prix) sur les 12 commandes). Lablabi (9 DT) est donc exclu. Une fonction d'agrégat ne peut pas être écrite directement dans WHERE : on passe par une sous-requête.

C4 – Défi : sous-requête + GROUP BY

Afficher les catégories dont le chiffre d'affaires est supérieur à la moyenne des chiffres d'affaires des catégories.

Requête :

```
SELECT categorie, SUM(prix*qte) AS ca
FROM COMMANDE
GROUP BY categorie
HAVING SUM(prix*qte) >
   (SELECT AVG(t.ca)
    FROM (SELECT SUM(prix*qte) AS ca
          FROM COMMANDE
          GROUP BY categorie) AS t);
```

Résultat :

categorie	ca
Plat	212.00

À expliquer : Trois niveaux : la table dérivée t contient une ligne par catégorie (son CA) ; AVG(t.ca) en fait la moyenne (93,75 DT) ; HAVING compare le CA de chaque pile à cette moyenne.

Remarque sur GROUP BY

MariaDB accepte parfois un SELECT contenant un champ absent du GROUP BY (mode sans ONLY_FULL_GROUP_BY), mais la valeur affichée pour ce champ est alors arbitraire et d'autres SGBD refusent la requête. Exiger la règle « tout champ non agrégé est dans le GROUP BY » évite cette erreur.

Partie D – Fonctions sur les dates

Rappel des fonctions :

- DATE(chaine) : extrait une date d'une chaîne au format DATE/DATETIME.
- DAY(date) : retourne le jour.
- MONTH(date) : retourne le mois.
- YEAR(date) : retourne l'année.
- NOW() : retourne la date et l'heure courantes.
- ADDDATE(date, n) : ajoute n jours (ou un intervalle) à une date.
- DATEDIFF(date1, date2) : différence en jours entre deux dates.

Question	Réponse :

D1. Afficher les commandes passées le 15 mars 2026.	`sql`
	`SELECT *`
	`FROM COMMANDE`
	`WHERE date_cmd = '2026-03-15';`

D2. Afficher, pour chaque commande, le numéro, le plat et le jour du mois de la commande.	`SELECT num, plat, DAY(date_cmd) AS jour FROM COMMANDE;`
D3. Afficher les commandes passées au mois de mars 2026	`SELECT * FROM COMMANDE WHERE MONTH(date_cmd) = 3 AND YEAR(date_cmd) = 2026;`
D4. Afficher, pour chaque année, le nombre de commandes.	`SELECT YEAR(date_cmd) AS annee, COUNT(*) AS nb_commandes FROM COMMANDE GROUP BY YEAR(date_cmd);`
D5. Afficher les commandes passées dans les 7 derniers jours par rapport à la date du jour (on suppose que la date du jour est le 15 avril 2027).	`SELECT * FROM COMMANDE WHERE date_cmd >= ADDDATE('2027-04-15', -7);` **Avec NOW() : WHERE date_cmd >= ADDDATE(NOW(), -7) (si la date système est le 15/04/2027).**
D6. Afficher, pour chaque commande, le nombre de jours écoulés depuis la commande jusqu'au 31 décembre 2027.	`SELECT num, plat, DATEDIFF('2027-12-31', date_cmd) AS jours_ecoules FROM COMMANDE;`
D7. Afficher les serveurs embauchés depuis plus de 5 ans par rapport au 1er janvier 2027.	`SELECT num, nom, salle FROM SERVEUR WHERE DATEDIFF('2027-01-01', date_embauche) > 5 * 365;` Ou plus simplement : WHERE date_embauche < ADDDATE('2027-01-01', -5*365);
D8. Afficher, pour chaque serveur, son nom et le nombre de commandes qu'il a prises au mois de mars 2026.	`SELECT s.nom, COUNT(*) AS nb_commandes FROM SERVEUR s, COMMANDE c WHERE s.num = c.numserveur AND MONTH(c.date_cmd) = 3 AND YEAR(c.date_cmd) = 2026 GROUP BY s.nom;`
D9. Afficher les catégories dont le chiffre d'affaires réalisé entre le 1er janvier 2027 et le 30 avril 2027 dépasse 50 DT.	`SELECT categorie, SUM(prix * qte) AS ca FROM COMMANDE WHERE date_cmd BETWEEN '2027-01-01' AND '2027-04-30' GROUP BY categorie HAVING SUM(prix * qte) > 50;`
D10. Expliquer pourquoi la requête suivante est refusée, puis la corriger. `SELECT num, plat, DAY(date_cmd) FROM COMMANDE WHERE DAY(date_cmd) = 15 GROUP BY YEAR(date_cmd);`	La clause SELECT contient num, plat et DAY(date_cmd) qui ne sont pas dans une fonction d'agrégat et ne figurent pas dans le GROUP BY (YEAR(date_cmd)). MariaDB refuse donc la requête. Correction (on veut le nombre de commandes du 15 de chaque année) :`SELECT YEAR(date_cmd) AS annee, COUNT(*) AS nb FROM COMMANDE WHERE DAY(date_cmd) = 15 GROUP BY YEAR(date_cmd);`
D11. Afficher les commandes dont la date est comprise entre le 1er janvier 2026 et le 31 décembre 2026, triées par date croissante.	`SELECT * FROM COMMANDE WHERE date_cmd BETWEEN '2026-01-01' AND '2026-12-31' ORDER BY date_cmd ASC;`

D12. Afficher, pour chaque serveur, la date de sa première commande et le nombre de jours entre cette première commande et le 31 décembre 2027.	`SELECT s.nom,`
	`  MIN(c.date_cmd) AS premiere_commande,`
	`  DATEDIFF('2027-12-31', MIN(c.date_cmd)) AS jours_depuis`
	`FROM SERVEUR s, COMMANDE c`
	`WHERE s.num = c.numserveur`
	`GROUP BY s.nom;`

D13. Afficher les commandes passées au cours du premier trimestre 2027 (janvier, février, mars).	`SELECT *`
	`FROM COMMANDE`
	`WHERE YEAR(date_cmd) = 2027`
	`  AND MONTH(date_cmd) IN (1, 2, 3);`

D14. Afficher le chiffre d'affaires par mois pour l'année 2026.	`SELECT MONTH(date_cmd) AS mois, SUM(prix * qte) AS ca`
	`FROM COMMANDE`
	`WHERE YEAR(date_cmd) = 2026`
	`GROUP BY MONTH(date_cmd)`
	`ORDER BY mois;`

D15. Afficher les serveurs dont la date d'embauche est postérieure au 1er janvier 2023, avec le nombre d'années écoulées jusqu'au 1er janvier 2027.	`SELECT nom, date_embauche,`
	`  DATEDIFF('2027-01-01', date_embauche) / 365 AS annees`
	`FROM SERVEUR`
	`WHERE date_embauche > '2023-01-01';`

Tableau récapitulatif des fonctions de date

Fonction	Rôle	Exemple
DATE()	Extrait une date d'une chaîne	DATE('2026-03-15 10:00:00') → 2026-03-15
DAY()	Jour du mois	DAY('2026-03-15') → 15
MONTH()	Mois	MONTH('2026-03-15') → 3
YEAR()	Année	YEAR('2026-03-15') → 2026
NOW()	Date et heure courantes	NOW()
ADDDATE()	Ajoute des jours	ADDDATE('2026-03-15', 7) → 2026-03-22
DATEDIFF()	Différence en jours	DATEDIFF('2026-03-22', '2026-03-15') → 7